CIT/P Portfolio Project Requirements

Complex IT Systems – Practice

September 17, 2026

This document describes the requirements to the three portfolio subprojects that are included in the mandatory Project Portfolio.

The overall goal of these portfolio subprojects is to build a system that allows multiple users to view information about, browse, search for, rate, and compare movies and actors. The system must keep track of users' search history and must support rating and bookmarking of movies.

Each portfolio subproject corresponds to one of the topics covered in the three main sections of the CIT/T course. The three subprojects will aim at, respectively, (1) building a data repository with embedded functionality for unified access, (2) developing services to access the repository and exposing these services through a well-defined interface, and (3) developing a web-based client that gives uses access to the provided services via a browser.

The database developed in the first subproject is based on data from the Internet Movie Database (IMDB). IMDB is an online database of information mainly related to movies and TV Series (but includes also information about other types of published material such as games and documentaries). It is the largest and most comprehensive movie database on the web and it includes more than 24 million titles (movies, TV-series and other) and more than 14.5 million personalities. IMDB was launched in 1990 and is now owned by Amazon.com. We will use a reduced version of IMDB's dataset. (See the *CITP Project Portfolio Source Data* found on Moodle for a description of the provided data.)

The solution to each subproject consists of models, scripts and programs that implement part of the system and a report that documents this. The results of subprojects 1 and 2 must be handed in as stand-alone project reports on the course's Moodle page. (The deadlines and instructions for handing in are described on Moodle.) The result of subproject 3 must be handed in via `eksamen.ruc.dk`. (The deadline is found on `study.ruc.dk` under the description of the course "Complex IT Systems – Practice".)

You should not include the answers to one subproject in the reports for subsequent subprojects. If you find the need to modify a previous subproject during the development of the current subproject, then explain those modifications in the report covering the current subproject.

Each of the three reports must document the parts of the system you implement. These reports must follow standard principles of academic and technical writing. Document

the (relevant) parts of the system (procedures, functions, methods, classes, components, tables, etc.) by analyzing their requirement and the context in which they are used, designing a solution, implementing the proposed design, evaluate the merits of the implementation, and reflecting on the result (thus following a standard approach to software engineering).

A report should not be a log documenting what you did when. Present the final product, instead of all the steps taken to develop it. A report should not just list the artifacts (procedures, functions, methods, classes, components, tables, etc.) that implement the system. Instead supplement the descriptions of these artifacts by motivating, explaining, and justifying the decisions that resulted in them. Avoid explaining solely by giving examples. Instead give general explanations and then (if needed) supplementary examples. The reports should be precise, consistent, concise, systematic, and readable. The reports must be formatted consistently. All sections must be numbered. All pages must be numbered. The report must contain a table of contents.

This document contains 17 pages.[1] It consists three chapters documenting the requirements to the three subprojects. We recommend that you read or skim the set of requirements for all three portfolio subprojects before getting started on the first.

[1] This project requirement is version 2026-09-17 14:30:13.

Contents

1 The database **4**

1-A Application design 4

1-B The Movie Data Model 4

1-C Framework Model 5

1-D Functionality 6

1-E Improving performance by indexing 9

1-F Testing using the IMDB database 9

2 The backend **10**

2-A Requirements 10

2-B API design 11

2-C Architecture 11

2-D Implementation 12

2-E Testing 13

3 The frontend **14**

3-A User interface design 14

3-B Presentation Layer 15

3-C Business Logic Layer 15

3-D Data Access Layer 15

3-E Functional requirements 16

3-F Non-functional requirements 17

Subproject 1

The database

The objective of Portfolio Subproject 1 is to deliver a database for the movie application and to set up key application functionality. The database should be designed around two partially independent data models: a Movie Data Model for storing movie information gathered from external sources, and a Framework Model for handling the application framework (users, bookmarks, local ratings, history). These two models must be integrated into a single database during implementation.

1-A Application design

Sketch a preliminary design of the application you intend to develop and the features that you aim to provide. Study the domain by considering the provided data and by browsing movie apps on the web. Based on this, develop your own ideas and describe these in brief. Develop and describe also a first sketch of how to show the history, ratings and bookmarks in your application. Give arguments for your design decisions and discuss and describe the implications on your data model. Since this is a preliminary design, it may obviously be subject to later changes — including later simplifications due to time limitations.

1-B The Movie Data Model

The data model for the movie data part must be designed so that the provided data can be represented. The provided data is described in the note *CITP Project Portfolio Source Data* that also explains how to get access to the data and load the data into a database in your own database server.

You can claim that the provided tables already comprise a movie data model as requested, but if you want to achieve a good design that doesn't violate common conventions concerning relational database design, a redesign is needed.

1-B.1 Create a data model to represent the provided data. Aim to design the model

independently of the original source structure, while ensuring that all information can still be represented. Use the database design methods and theory covered in the database section of the CITT course as guidance. Be sure to document both intermediate and final versions of your models, consider alternative designs, and justify the decisions you make. Present an ER diagram of your final model using the notation specified in the DB-book.

1-B.2 Implement the model developed in 1-B.1 as a relational database in PostgreSQL. Start by creating a new database and importing all source data as outlined in *CITP Project Portfolio Source Data*. Then, define the tables that implement your 1-B.1 model and transfer the data from the imported source tables into these new structures. After the data has been successfully migrated, remove the original source tables. (Notice that the values of `nconst` are needed by the frontend; these should be available in the data model. See 3-D.2.)

Collect all your commands for creating tables, distributing data and deleting source data in a single SQL script called `B2_build_movie_db.sql`. Make this script such that it can be modified and executed repeatedly with `psql` until you are satisfied with the result and have tested that all data is in the right place. To make a script so that it can be executed repeatedly, simply include "wipe-out" of all the old stuff — either by `dropping` things to start with or by using `create or replace` rather than just `create`. The major part of your SQL script will be `create table` and `insert` statements. (Maybe like this: `insert into xxx select ... from ...`). However, you may also need to include `do`-blocks in your SQL script, for instance if you need to iterate through a source table and split data. When you are done, generate a *reverse engineered ER diagram*[1] and include this as well as your SQL script in your documentation.

If your considerations from Section 1-A calls for changes to the Movie-data model, you can just include these changes in 1-B.1 and 1-B.2. Just remember to add a note about what the changes are.

1-C Framework Model

In essence, the framework model should support the following: registration of users of your application and, for users individually, storage of search history, storage of rating history, bookmarking of titles and names (personalities). Feel free to extend the Framework with your own additions (such as support addition of personal notes to titles).

1-C.1 Develop a data model that is appropriate for the purpose of the framework. Provide (and include in your report) an ER diagram of your framework model. Indicate how your Framework Model connects to the Movie Data Model by

[1] Open your database in Navicat and right click on the schema (probably called "public"). Then select "Reverse Schema to Model ..." and click the Diagram tab (or double click the diagram line). You can edit the diagram by moving boxes and connectors, if you would like to change the layout. When you are done, you can save it or just copy the graphics by taking a screenshot and paste this into your report.

including the entities from your Movie Data Model in 1-B.1 that are directly connected with relationships from the Framework Model. It must be clear how the two models combine. Optionally, provide a complete ER diagram with all entities and relationships from both models. Use the ER notation described in the DB-book.

1-C.2 Implement the Framework Model. Write an SQL script that adds the Framework Model to an already created Movie Data Model database (the result from 1-B.2). Name this script `C2_build_framework_db.sql`. Since there is no data for the Framework part yet, the script will be mostly `create table` statements. Generate a reverse engineered ER-diagram of your full (Movie Data + Framework) database and include this as well as your SQL script in your documentation.

1-D Functionality

In addition to modeling and implementing the database, an essential part of this subproject is to develop core functionality that the data layer can expose for use by the service layer. The objective is to provide this functionality through an Application Programming Interface (API), consisting of a set of functions and procedures implemented in PostgreSQL.

The initial requirements focus on three areas: framework support, a basic search function, and functionality for rating titles.

1-D.1 **Basic framework functionality:** Consider what is needed to support the framework and develop functions for that. You will need functions for managing users and for bookmarking names and titles. You could also consider developing functions for adding notes to titles and names and for retrieving bookmarks as well as search history and rating history for users.

1-D.2 **Simple search:** Develop a simple search function called `string_search()`. This function should, given a search string S as parameter, find all movies where S is a substring of the title or a substring of the plot description. For the movies found, return id and title (`tconst` and `primarytitle`, if you kept the attribute names from the provided dataset). Make sure to bring the framework into play, such that the search history is updated as a side effect of the call of the search function.

1-D.3 **Title rating:** Introduce functionality for rating by a function, called `rate()`, that takes a title and a rate as an integer value between 1 and 10, where 10 is best.[2] The function should update the (average) rating appropriately taking the new vote into consideration. Make sure to bring the framework into play, such that the rating history is updated as a side effect of the call of the rate function. Also make sure to treat multiple calls of `rate()` by the same user

[2] See the detailed interpretation of IMDB ratings at help.imdb.com/article/imdb/track-movies-tv/ratings-faq/G67Y87TFYYP6TWAV.

consistently. This could be for instance by ignoring or blocking an attempt to rate, in case a rating of the same movie by the same user is already registered. Alternatively, an update with the new rate can be preceded by a "redrawing" of the previous rating, recalculating the average rating appropriately.

The following section extends the concept of search beyond basic string matching. Update the database and implement a set of functions and procedures that fulfill the specified requirements. When doing so, consider and discuss possible alternatives, justify your design decisions, and, where relevant, refer to applicable methodology and theory. Ensure to bring the framework into play where appropriate.

1-D.4 Structured string search: Create a search function that accepts four string parameters and name the function `structured_string_search()`. The function should:

- Return movies where the four provided parameters match the four fields: title, plot, characters, and person names, respectively.
- Matching should be case-insensitive.
- Parameter values should be treated as substrings, meaning a match occurs if the value is contained anywhere in the corresponding column.

For each movie found, the function should return its ID (`tconst` in the source data) and title (`primarytitle`). Make sure to bring the framework into play, such that the search history is stored as a side effect of the call of the search function.

1-D.5 Finding names: The above search functions are focused on finding titles. Try to add to these by developing one or two functions aimed at finding names (of, for instance, actors).

1-D.6 Finding co-players: Make a function that, given the name of an actor, will return a list of actors that are the most frequent co-players to the given actor. For the actors found, return their `nconst`, `primaryname` and the frequency (number of titles in which they have co-played).

Hint: You may, for this as well as for other purposes, find a view helpful to make query expressions easier (to express and to read). An example of such a view could be one that collects the most important columns from title, principals and name in a single virtual table.

1-D.7 Name rating: Derive a rating of names (just actors or all names, as you prefer) based on ratings of the titles they are related to. Modify the database to store also these name ratings. Make sure to give higher influence to titles with more votes in the calculation. You can do this by calculating a weighted average of the `averagerating` for the titles, where the `numvotes` is used as weight.

1-D.8 Popular actors: Propose and implement a function that utilizes the name ratings. One option is a function that takes a movie as input and returns the list of

actors in that movie, ordered by decreasing popularity. Another could be a similar function that takes an actor and lists the co-players in order of decreasing popularity.

1-D.9 **Similar movies:** Discuss and suggest a notion of similarity among movies. Design and implement a function that, given a movie as input, will provide a list of other movies that are similar.

1-D.10 **Frequent person words:** The `wi` table provides an inverted index for titles using the four columns: `primarytitle`, `plot` and, from persons involved in the title, `characters` and `primaryname`. So, given a title, we can from `wi` get a lot of words, that are somehow characteristic for the title. To retrieve a list of words that are characteristic for a person we can do the following: find the titles the person has been involved in, and find all words associated with these titles (using `wi`). To get a list of unique words, you can just group by word in an aggregation by `count()`. Thereby you'll get a list of words together with their frequencies in all titles the person has been involved in. Use this principle in a function `person_words()` that takes a person name as parameter and returns a list of words in decreasing frequency order, limited to fixed length (e.g. 10). Optionally, add a parameter to the function to set a maximum for the length of the list. You can consider the frequency to be a weight, where higher weight means more importance in the characteristics of the person.

1-D.11 **Exact-match querying:** Introduce an exact-match querying function that takes one or more keywords as arguments and returns titles that match all of these. Use the inverted index `wi` for this purpose. You can find inspiration on how to do that in the slides on Textual Data and IR.

1-D.12 **Best-match querying:** Develop a refined function similar to 1-D.11, but now with a "best-match" ranking and ordering of objects in the answer. A best-match ranking simply means: the more keywords that match, the higher the rank. Titles in the answer should be ordered by decreasing rank. See also the Textual Data and IR slides for hints.

1-D.13 **Word-to-words querying:** An alternative, to providing search results as ranked lists of posts, is to provide answers in the form of ranked lists of words. These would then be *weighted keyword lists* with weights indicating relevance to the query. Develop functionality to provide such lists as answer. One option to do this is the following: (1) Evaluate the keyword query and derive the set of all matching titles, (2) count word frequencies over all matching titles (for all matching titles collect the words they are indexed by in the inverted index `wi`), and (3) provide the most frequent words (in decreasing order) as an answer (the frequency is thus the weight here).

1-D.14 **Weighted querying:** Build a new inverted index for weighted indexing similar to the `wi` index, but now with added weights. A weight for an entry in the index should indicate the relevance of the title to the word. As weighting strategy, a good choice would probably be a variant of TFIDF.

Develop a refined function similar to 1-D.12, but now with a ranking based on relevance weighting provided by the weighted indexing.

1-D.15 **Own ideas [Optional]:** If you have some ideas of your own, you can plug them in here.

1-E Improving performance by indexing

A key advantage of using a relational database is that query performance can be dynamically optimized at any stage of development and tailored to actual system usage, particularly for the most frequent or critical queries — even after deployment. Such performance improvements can be made by adding or adjusting indexes on database tables, without requiring changes to other parts of the system.

1-E.1 Consider the extensions developed under Section 1-D and discuss or explain what may potentially provide significant performance improvements. Describe how your database is indexed. (Observe that this concerns database indexing, not textual inverted indexing — even though the latter may build on the former.)

1-F Testing using the IMDB database

Demonstrate by examples that the results of Section 1-D work as intended. Write a single SQL script that activates all the written functions and procedures and, for those that modify data, add selections to show before and after for the modifications. In this subproject you need only to proof by examples that you code is runnable. A more elaborate approach to testing is an issue in Subproject 2. Generate an output file from running your test script file. (See descriptions in assignment 1 and 2 on how to do this.)

Subproject 2

The backend

The objective of Subproject 2 is to make the database built in Subproject 1 available to client applications through a RESTful web service. The frontend developed in Subproject 3 will be the first such client, and the interface you design here is the contract it depends on.

The backend is developed in TypeScript on Node.js. The web service is implemented with the Express framework, and the database is accessed through the node-postgres library. Tests are written with Jest or Vitest, and endpoint tests with supertest.

Hand in a report documenting the requirements, the design of the interface, the architecture, the implementation and the tests, together with the source code.

2-A Requirements

Some requirements are given below. The remainder you derive yourself from the application you intend to build.

2-A.1 **Functionality:** The functionality developed in Sections 1-C and 1-D must be reachable through the API. This includes user management, bookmarking, search history and rating history from the framework model, and the search, rating, co-player, similarity and word-frequency functionality developed in the database. IMDB data is read-only, so the API must not offer creation, update or deletion of it. Framework data must support the full range of create, read, update and delete operations.

2-A.2 **Self-descriptive responses:** Every response must identify the resources it refers to by URI rather than by bare identifier. This applies to individual resources and to every element of a list.

2-A.3 **Paged listings:** Every operation returning a list must be paged. The client may state a page size. If none is given, or the requested size exceeds the maximum, a default is applied. A page must include links to the previous and the next page where these exist.

2-A.4 Non-functional requirements: No operation may return an unbounded result set. Passwords must be stored only as salted hashes produced by an established password hashing function. Error responses must not disclose stack traces, SQL text or database identifiers. The application must connect to the database with a role that holds only the privileges it needs.

2-A.5 Requirements you derive: Revise the application design from Section 1-A and express it as use cases or user stories. Derive from these the operations your API must offer beyond those required above, and the information each must return. Document them and justify them against the application you intend to build.

2-B API design

2-B.1 Resource model: Identify the resources your API exposes and specify, for each, the URIs by which it is addressed, the HTTP methods it supports, and the status codes each operation may return. Explain which entities of your data model became resources and which did not, and how relationships are expressed, whether as sub-resources or as query parameters.

2-B.2 Representations: Specify the JSON representation of each resource: its fields, their types, and how nested and related resources appear. Specify also the representation of errors, which must be uniform across the API. This specification is the contract on which Subproject 3 depends, so document it precisely.

2-B.3 Design rationale:. Explain how your interface satisfies the requirements of Section 2-A, and justify the decisions you made where more than one design was possible.

2-C Architecture

2-C.1 Architecture: Design the architecture of the backend. Define the elements the system is composed of, the responsibility of each, and the dependencies permitted between them. The architecture may have more than one level, in which case state the elements at each level and the rules that apply there.

Describe the architectural style or styles you have applied, or explain how your architecture departs from any established style. A layered architecture and the Model-View-Controller pattern are two possibilities, but neither is required.

Document the architecture, justify your decisions, and include the directory structure of your project, explaining how it reflects the architecture.

2-C.2 Allocation of logic: Decide, for each rule the application enforces, whether it belongs in a PostgreSQL function or in the application code. Several such rules were implemented in the database in Subproject 1. Justify each allocation with reference to performance, testability and maintainability.

2-C.3 **Data access:** Decide how access to the database is separated from the handling of requests, and document the interface this separation produces. Explain what the parts above it can and cannot see.

2-C.4 **Mapping:** Decide how a row returned by the driver becomes a representation designed in Section 2-B. Document how columns map to fields, how nullable columns are represented, and how joined data becomes nested or flat.

2-C.5 **Supply of dependencies:** Express provides no dependency injection container. Decide how a component obtains what it depends on, such as how the code that queries the database obtains the connection pool, and justify the mechanism you chose.

2-C.6 **Authentication mechanism:** Decide how the identity of a user accompanies a request, given that RESTful services are stateless. Justify your choice and state its limitations.

2-C.7 **Cross-cutting concerns:** Decide which concerns are handled by middleware rather than within individual request handlers, and in what order the middleware runs. Explain the consequences of that ordering.

2-D Implementation

2-D.1 Implement the interface designed in Section 2-B, following the architecture designed in Section 2-C.

2-D.2 Requests must be routed so that the structure of the routing reflects the resource model. Path parameters and query parameters must be validated before use, and the appropriate status code returned when validation fails.

2-D.3 Database connections must be drawn from a pool rather than opened for each request. All values originating from a client must be passed as query parameters and never assembled into SQL by string concatenation.

2-D.4 Operations that must succeed or fail as a unit must be executed on a single client within an explicit transaction.

2-D.5 Unexpected failures must be handled centrally, so that error handling is not repeated in every request handler, and must produce the error representation specified in 2-B.2.

2-D.6 Implement user management and the authentication mechanism chosen in 2-C.6, so that search history, ratings and bookmarks belong to individual users. In Express, verifying a caller's identity and deciding what that caller may do are both expressed as middleware.

2-D.7 The frontend of Subproject 3 will be served from a different origin than this backend, so a cross-origin resource sharing policy must be configured and documented.

2-E Testing

Test each part of the implementation, at unit level and in integration. Exhaustive testing is not expected; demonstrate instead how each kind of component is tested, and test one representative of each kind.

2-E.1 Test the logic that does not touch the database, such as the mapping from rows to representations.

2-E.2 Test the code that accesses the database, against a test database or within a transaction that is rolled back, so that repeated runs give the same result.

2-E.3 Test the API over HTTP, without starting a server by hand. Demonstrate at least a read of IMDB data, a write of framework data, a paged listing, and a request that must be rejected as unauthorised.

2-E.4 Show how your tests demonstrate that the requirements of Section 2-A are met.

Subproject 3

The frontend

The goal of Portfolio Subproject 3 is to design and implement a frontend that provides a graphical user interface to the movie application. This frontend must run in a user's browser and it must communicate with the backend that you have implemented as part of Portfolio Subproject 2.

You should structure the frontend so that it consists of decoupled components organized into well-defined parts or layers with separate responsibilities. For example, the frontend may be internally organized into a Presentation Layer responsible for how data is presented to the user, a Business Logic Layer containing any logic independent of the presentation layer and data access, and a Data Access Layer implementing the communication with the backend web service.

Besides communicating with your own backend, the frontend must fetch pictures of persons from The Movie Database (TMDB), by making requests similar to those of the assignment on The Movie Database found on Moodle under frontend development. (See 3-D.2.)

3-A User interface design

You must design and document the user interface and the information architecture of the frontend.

You should design the user interface such that it complies with Jacob Nielsen's 10 Usability Heuristics for User Interface Design.[1]

3-A.1 Design the intended site structure of the frontend. Document this design with a site structure diagram that shows the different "pages" or "views" that the frontend provides and how the user can transition and navigate between them.[2]

[1] See www.nngroup.com/articles/ten-usability-heuristics/.
[2] See www.bluehost.com/resources/website-structuring/.

3-A.2 Design the intended user interfaces of the individual "pages" and components of the frontend. Document them using wireframe models.[3]

3-A.3 Design and document the routes that can be used to navigate the application.

3-B Presentation Layer

The actual graphical user interface must be implemented in JavaScript or TypeScript using the React library and Bootstrap (but see 3-F.1). It should implement the design from Section 3-A.

3-B.1 Document React components that you have developed and their roles; document the component hierarchy.

3-B.2 Describe React patterns, principles, and techniques that you have used (e.g., prop drilling, lifted state, `useState`, `useEffect`, `useContext`, and any other hooks applied, DOM router, controlled components, and how communication with backend is implemented).

3-B.3 Describe the purpose of third-party JavaScript or TypeScript, React, or Bootstrap libraries or components that your application uses.

3-B.4 Your solution must implement the user interface with (React) Bootstrap. Describe how (and if) Bootstrap helps you implement the user-interface design from Section 3-A.

3-C Business Logic Layer

You may chose to represent data in the frontend differently from the representation in the backend. Such an internal representation may, for example, reduce coupling to the backend and may ease combining data from different sources or servers. (See Section 3-D.)

3-C.1 Describe the internal representation of data as objects and describe any classes or functions that you have implemented to help maintain these representations.

3-C.2 Describe other helper functions for manipulating data in the frontend.

3-D Data Access Layer

A data access layer may provide an interface to the backend: Instead of connecting to the web service offered by the backend directly from the presentation layer (i.e., React components), you may implement a *data service* that defines an internal interface (for example, a set of functions or one or more classes) that the business logic layer can use to communicate (indirectly) with the backend.

[3] See `en.m.wikipedia.org/wiki/Website_wireframe`.

This architecture prevents future changes in the backend from propagating into the business logic layer and the presentation layer of the frontend.

The data service should communicate asynchronously with the backend. If the frontend is written in TypeScript, consider reusing the data transfer object types defined for the backend web service (see 2-A.3) as the return types of this layer, so that a change to the outer facing contract is caught at compile time on both sides. If you do so, describe how the shared types are organized.

3-D.1 Show where and how data access is implemented and document the functionality it provides.

3-D.2 The frontend must fetch pictures of persons from The Movie Database (TMDB), following the approach outlined below. (These pictures should, of course, be used where appropriate by the Presentation Layer.)

The attribute `nconst` in table `name_basics` uniquely identifies a person. We can use this IMDB-specific ID to find the corresponding person in TMDB, by issuing a request to TMDB with parameter `external_source=imdb_id`, as follows (where *N* is this IMDB-specific ID and *K* is your TMDB API key).

`https://api.themoviedb.org/3/find/`*N*`?external_source=imdb_id&api_key=`*K*

The result of this request is a JSON-encoded object *r* whose `person_results` field contains an array of *person objects*. Therefore, we can get the TMDB-specific ID of the (first) person of this response by the expression

```
$r$.person_results[0].id
```

(For example, Steven Spielberg has `nconst` `nm0000229` in IMDB and ID 488 in TMDB.) This TMDB-specific ID gives access to profiles, and therefore `file_paths` and URLs, of pictures of a person on The Movie Database.

3-E Functional requirements

The frontend must provide a user interface to the movie application. The user interface must be designed such that it appears consistent and coherent.

3-E.1 Your solution should be a single-page application. (If you divert from this design, then explain where and why.)

3-E.2 The frontend must provide a navigation bar to transition between different types of content. It may also provide links to transition between content where you find it relevant.

3-E.3 Content with listed information must be provided with pagination when possible.

3-E.4 Content that is uniquely identifiable (for example names and titles) must be accessible via a route (so that, for example, the user may browser-bookmark names and titles).

3-E.5 Features supported by the framework model should also be available from the frontend, such as registration of users and bookmarking of titles and people. (See Section 1-C.)

3-E.6 The functional requirements listed under Section 1-D should be supported by the frontend. In particular, the frontend should support searching for movies and people, and rating titles.

3-F Non-functional requirements

3-F.1 You should implement the frontend primarily using React, HTML, CSS, and (React) Bootstrap. Since the practice course teaches TypeScript from the outset and the backend of Subproject 2 is written in TypeScript, we recommend that the frontend be written in TypeScript as well, so that the two halves of the system share one language and, if you wish, one set of type declarations for the outer facing contract. A frontend written in plain JavaScript is still accepted.

Whichever language you choose, document the consequences of that choice for the maintainability and testability of your code. (Notice that we do not evaluate a solution more favorably merely because it is implemented in TypeScript.)

You are, however, allowed to implement part (but not all) of the frontend in TypeScript. In that case, you should document advantages and disadvantages resulting from the use of TypeScript over JavaScript. (Notice that we do not evaluate a solution more favorably just because part of it is implemented in TypeScript.)

You are also allowed to encode part (but not all) of the styling of HTML using other CSS frameworks than (React) Bootstrap. In that case, you must document significant differences, advantages, and disadvantages between the two frameworks and you must motivate the choice of a different framework. (Again, we do not evaluate a solution more favorably just because it uses a different CSS framework.)

3-F.2 You should aim for the implementation being modular, scalable, maintainable, testable, fault tolerant, and extendable. In particular, you should split the solution into separate (JavaScript and JSX or TypeScript and TSX) files with well-defined interfaces, and export only relevant components, functions, variables, constants, and classes from these files.